SEAT NO. CT- 23025

NED UNIVERSITY OF ENGINEERING & TECHNOLOGY

THIRD YEAR (COMPUTER SCIENCE/ COMPUTER SCIENCE SPECIALIZATION IN CYBER SECURITY/ GAMING AND ANIMATION)

FALL SEMESTER EXAMINATIONS 2025

BATCH 2023

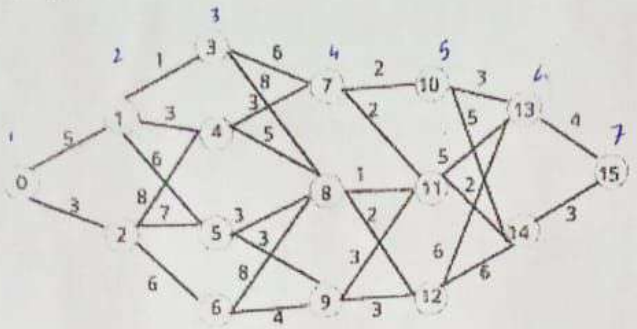
Time: 3 Hours | **Dated: 03-DEC-25**

Max.Marks:60

Design and Analysis of Algorithms – CT-363

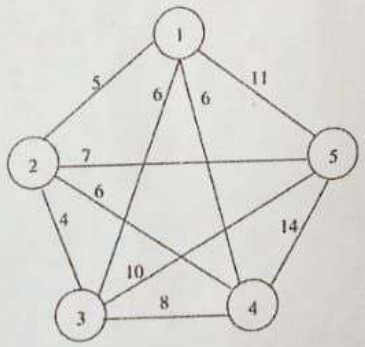
Q1) **CL0 3 12 marks**

a) Apply Dynamic Programming approach and find the minimum path of multistage graph from source 0 to 15.



Q2) **CL0 3 12 marks**

a) Make use of Branch and Bound strategy to solve traveling salesman problem.



Q3) **CL0 2 12 marks**

A professor is grading midterms for a class of 12 students. The raw scores (out of 100) are: [85, 92, 78, 95, 67, 88, 76, 91, 83, 69, 94, 81]. The professor wants to sort them in **descending** order to award the top performers.

- **Discuss** how Tim Sort will be use to sort the scores. Assume Run Size = 6, perform detail working on 1st block of run and then show the intermediate merge steps.
- **Extend** Jump search to find the position where a new score of 88 would fit (insertion point for maintaining order). How many students scored highest.

P.T.O

Q4)	CL0 2 8+4 marks

a) A TV network has ad slots from 1-12 PM. 10 advertisers bid with time intervals and profits:

Add no: (Start time, End Time, Profit)

Ad1: (1, 3, 50), Ad2: (2, 5, 70), Ad3: (4, 6, 40), Ad4: (5, 7, 90), Ad5: (6, 8, 60) Ad6: (3, 9, 120), Ad7: (8, 10, 80), Ad8: (9, 11, 110), Ad9: (7, 12, 100), Ad10: (0, 2, 30)

Discuss the working of greedy algorithm, sort data by profit per unit time (density = profit / duration), then select non-overlapping highest density first. What is the total profit and selected ads?

b) Explain how dynamic programming approaches (Memorization, Tabular) will help in reducing the recursive calls for Fibonacci series. Provide working of both approaches by using Fib (5)

Q5)	CL0 2 8+4 marks

Dry run step-by-step. Draw recursive tree, **find** recurrence relation, also **find** complexity for the following codes .

a) A = [2, 4, 6, 8, 10, 12, 14], start = 1, end = 7	b) x = 7

Algorithm X5(A, start, end):
  if start == end:
  return A[start]
 	
  mid1 = ⌊(2*start + end)/3⌋
  mid2 = ⌊(start + 2*end)/3⌋
 	
  z1 = X5(A, start, mid1)
  z2 = X5(A, mid1+1, mid2)
 	
  t = 0
  for i = start to end:
  for j = start to mid2:
  t = t + (A[j] % A[i])
 	
  z3 = X5(A, mid2+1, end)
 	
  return z1 + z2 + z3 + t
	Algorithm Q2(x):
	  if x <= 2:
	  return x
	  s = 0
	  for i = 1 to x:
	  s = s + i
	  return s + Q2(x-2)